

GoRyde

Complete System Design & Implementation Plan

Real-Time Urban Ride Dispatch & Mobility Orchestration System
UrbanGlide Mobility

Document purpose: This document freezes the agreed architecture, service boundaries, database design, API contracts, frontend structure, real-time architecture, security model, development sequence, review scope, testing strategy, and deployment plan for GoRyde.

1. Project Overview

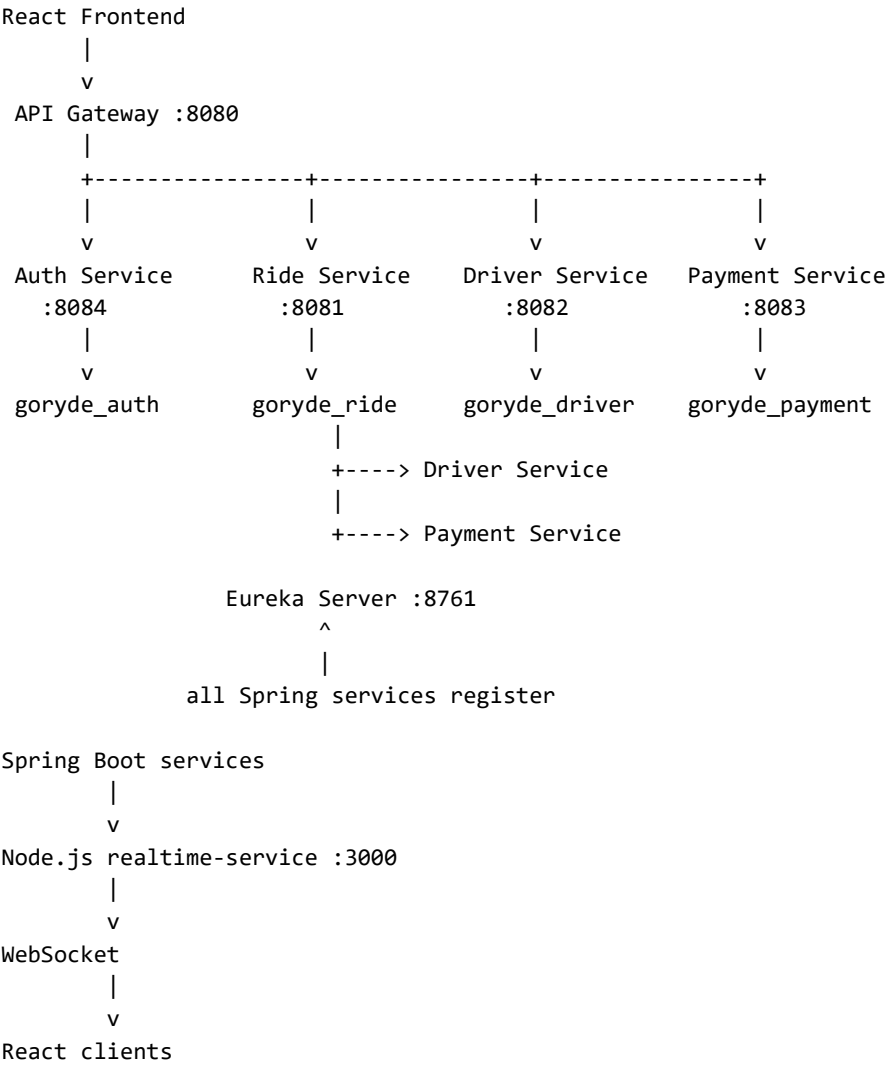
GoRyde is a real-time urban ride dispatch and mobility orchestration platform for UrbanGlide Mobility. It connects passengers, drivers, and administrators through a React web application backed by Spring Boot microservices, service discovery, an API Gateway, MySQL databases, and a Node.js real-time communication layer.

- Passengers can register, authenticate, book rides, track rides, view ride history, and manage payments.
- Drivers can manage availability, receive ride requests, accept/reject rides, execute active rides, and view earnings/history.
- Administrators can monitor users, drivers, rides, and payments.
- Ride Service acts as the primary business orchestrator for booking, driver assignment, ride lifecycle, and payment initiation.
- Eureka provides service discovery and the API Gateway provides a single entry point.
- Node.js provides WebSocket-based live updates without mixing real-time transport responsibilities into the transactional Spring Boot services.

2. Technology Stack

Layer	Technology
Frontend	React 19, Vite 8, Tailwind CSS 4, React Router DOM 7
Frontend utilities	Axios, React Hot Toast, React Icons, Lucide React, Swiper, AOS, React CountUp, React Flow
Backend	Java 17, Spring Boot 3.5.16
Security	Spring Security, JWT
Persistence	Spring Data JPA, MySQL
Service discovery	Netflix Eureka
Gateway	Spring Cloud Gateway
Inter-service communication	OpenFeign / service-discovery based HTTP
Real-time	Node.js + WebSocket
Testing	JUnit, Spring Boot Test, integration/API testing, frontend testing
Deployment	Docker / Docker Compose, with production deployment planned after local validation

3. High-Level Architecture

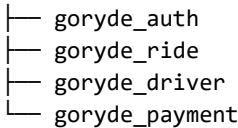


4. Service Responsibilities

Service	Owns	Does Not Own
Auth Service	Users, credentials, roles, authentication, JWT	Ride lifecycle, driver availability, payments
Ride Service	Rides, ride lifecycle, fare calculation, dispatch orchestration	Driver master data or payment persistence
Driver Service	Drivers, vehicles, availability, driver location, driver workflow	Ride master record or payment record
Payment Service	Payments, transaction status, payment lifecycle	Ride assignment or driver management
API Gateway	Routing, centralized entry, gateway-level security policies	Business-domain logic
Eureka Server	Service registry/discovery	Business data
Node.js realtime-service	WebSocket connections and live event delivery	Transactional business persistence

5. Database Architecture

MySQL



Each business service owns its database/schema. Cross-service identifiers are logical references rather than database-level foreign keys. This prevents tight database coupling between microservices.

5.1 Core Data Model

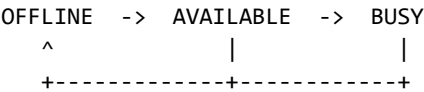
Service	Core Tables / Fields
Auth	users: user_id, name, email, password_hash, role, created_at, updated_at
Ride	rides: ride_id, passenger_id, driver_id, pickup_location, drop_location, distance, duration, fare, status, requested_at, completed_at
Driver	drivers: driver_id, user_id, name, vehicle_details, availability, latitude, longitude, created_at, updated_at
Payment	payments: payment_id, ride_id, passenger_id, amount, status, transaction_reference, created_at, updated_at

6. Roles and State Machines

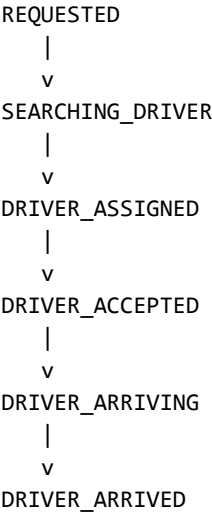
6.1 Roles

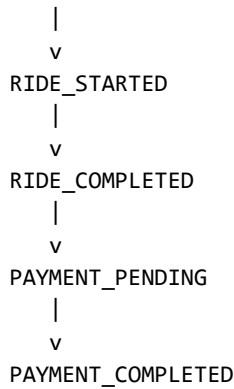
- PASSENGER
- DRIVER
- ADMIN

6.2 Driver Availability



6.3 Ride Lifecycle





Cancellation is allowed from appropriate pre-completion states:
REQUESTED / SEARCHING_DRIVER / DRIVER_ASSIGNED / DRIVER_ACCEPTED
-> CANCELLED

6.4 Payment Lifecycle

PENDING -> PROCESSING -> SUCCESS
 \-> FAILED
SUCCESS -> REFUNDED (when refund is applicable)

7. Core Ride Flow

1. Passenger authenticates and obtains a JWT.
2. Passenger submits POST /api/rides through the Gateway.
3. Ride Service validates the request and creates a ride in REQUESTED state.
4. Ride Service changes the ride to SEARCHING_DRIVER.
5. Ride Service calls Driver Service to identify an available suitable driver.
6. Driver Service filters AVAILABLE drivers and selects the nearest suitable driver.
7. Ride Service assigns the driver and changes the ride to DRIVER_ASSIGNED.
8. A real-time event is sent to relevant clients.
9. Driver accepts the request; the ride moves to DRIVER_ACCEPTED.
10. Driver progresses through ARRIVING and ARRIVED states.
11. Driver starts the ride; ride moves to RIDE_STARTED.
12. Driver completes the ride; ride moves to RIDE_COMPLETED.
13. Ride Service initiates payment through Payment Service.
14. Payment is processed and the final ride/payment state is delivered to the passenger.

8. Driver Matching Design

The first implementation uses a simple and explainable nearest-driver algorithm. It is intentionally suitable for the initial academic/project implementation and can later be optimized.

15. Retrieve drivers whose availability is AVAILABLE.
16. Validate that a driver has usable location data and an eligible vehicle.
17. Calculate distance from pickup point to driver location.
18. Rank candidates by distance.
19. Select the nearest suitable driver.
20. Reserve/mark the selected driver as BUSY only through controlled service logic.
21. Return the selected driver identifier to Ride Service.

Future optimization can add geographic indexing, search radius, driver ranking, vehicle type, demand balancing, and timeout/retry logic.

9. Fare Calculation

$\text{Fare} = \text{Base Fare} + (\text{Distance} \times \text{Per-KM Rate}) + (\text{Duration} \times \text{Per-Minute Rate})$

The initial fare model is deterministic and belongs to Ride Service. Fare-related values should be configurable rather than hard-coded throughout controllers.

10. REST API Contract

10.1 Auth Service

Method	Endpoint	Purpose
POST	/api/auth/register	Register a user
POST	/api/auth/login	Authenticate and return JWT
GET	/api/auth/me	Return authenticated user information

10.2 Ride Service

Method	Endpoint	Purpose
POST	/api/rides	Create/book a ride
GET	/api/rides/{rideId}	Get ride details
GET	/api/rides/my-rides	Get passenger ride history
POST	/api/rides/{rideId}/cancel	Cancel a ride
POST	/api/rides/{rideId}/start	Start a ride
POST	/api/rides/{rideId}/complete	Complete a ride

10.3 Driver Service

Method	Endpoint	Purpose
POST	/api/drivers	Create driver
GET	/api/drivers/{driverId}	Get driver details
PUT	/api/drivers/{driverId}	Update driver
PATCH	/api/drivers/{driverId}/availability	Change availability
GET	/api/drivers/available	Find available drivers
POST	/api/drivers/{driverId}/accept	Accept ride request
POST	/api/drivers/{driverId}/reject	Reject ride request

10.4 Payment Service

Method	Endpoint	Purpose
POST	/api/payments	Create/process payment
GET	/api/payments/{paymentId}	Get payment
GET	/api/payments/ride/{rideId}	Get payment for ride
GET	/api/payments/my-payments	Get passenger payments

11. API Gateway Routes

Incoming Path	Target Service
/api/auth/**	AUTH-SERVICE

/api/rides/**	RIDE-SERVICE
/api/drivers/**	DRIVER-SERVICE
/api/payments/**	PAYMENT-SERVICE

The Gateway is the external entry point. Clients should not need to know the internal ports of individual services.

12. JWT Security Design

- Authentication occurs through Auth Service.
- A successful login produces a signed JWT containing the required identity/role claims.
- The frontend stores and sends the token through the centralized Axios configuration.
- Gateway and protected services validate the token according to the agreed security design.
- Authorization is role-based: passenger, driver, and admin endpoints are restricted to appropriate roles.
- Business ownership checks remain inside domain services; JWT authentication alone is not sufficient for resource ownership.

Example: a passenger must not be allowed to read or modify another passenger's ride merely because the request contains a valid JWT.

13. Inter-Service Communication

Ride Service is the main orchestrator for the business workflow.

Ride Service -> Driver Service

Ride Service -> Payment Service

OpenFeign with Eureka service discovery is the planned implementation for synchronous internal HTTP communication. Controllers should not directly construct or manage inter-service HTTP calls.

14. Real-Time Architecture

Transactional actions use REST. Live status changes use WebSocket.

```

Spring Boot services
    |
    | event/update
    v
Node.js realtime-service
    |
    | WebSocket
    v
Passenger / Driver React clients

```

- Ride assigned -> passenger/driver receives live update.
- Driver accepts -> passenger receives live update.
- Driver location/status changes -> relevant passenger receives update.
- Ride started/completed -> passenger receives live update.
- Payment result -> passenger receives live update.

Node.js is a transport/event-delivery layer, not a replacement for the Spring Boot business services.

15. Frontend Information Architecture

```
src/
├── assets/
├── components/
│   ├── common/
│   ├── layout/
│   ├── navigation/
│   ├── ride/
│   ├── driver/
│   ├── payment/
│   └── dashboard/
├── pages/
│   ├── public/
│   ├── auth/
│   ├── passenger/
│   ├── driver/
│   └── admin/
├── routes/
│   ├── AppRoutes.jsx
│   ├── ProtectedRoute.jsx
│   └── RoleRoute.jsx
├── services/
│   ├── api/
│   ├── auth/
│   ├── rides/
│   ├── drivers/
│   └── payments/
├── context/
│   ├── AuthContext.jsx
│   └── RideContext.jsx
├── hooks/
├── utils/
├── App.jsx
├── main.jsx
└── index.css
```

15.1 Frontend Pages

Area	Pages
Public	Home, About, How It Works, Contact, Login, Register
Passenger	Dashboard, Book Ride, Ride Tracking, Ride History, Payments, Profile
Driver	Dashboard, Ride Requests, Active Ride, Ride History, Earnings, Profile
Admin	Dashboard, Users, Drivers, Rides, Payments

15.2 Frontend Rules

- Use centralized Axios configuration and interceptors.
- Attach JWT centrally rather than manually in every API call.
- Use ProtectedRoute and RoleRoute for access control.
- Use reusable components for repeated ride, driver, payment, dashboard, and layout UI.
- Use React Context plus local component state initially; introduce Redux only if complexity actually requires it.
- Use a WebSocket hook such as useWebSocket.js for real-time updates.

16. Repository Structure

```
GoRyde/
├── frontend/
│   └── goryde-web/
├── backend/
│   ├── eureka-server/
│   ├── api-gateway/
│   ├── auth-service/
│   ├── ride-service/
│   ├── driver-service/
│   └── payment-service/
├── realtime/
│   └── realtime-service/
├── docs/
│   ├── architecture/
│   ├── database/
│   ├── api/
│   ├── diagrams/
│   └── reviews/
├── postman/
├── docker/
├── .gitignore
├── README.md
└── docker-compose.yml
```

17. Backend Package Structure

```
config/
controller/
service/
repository/
entity/
dto/
enums/
exception/
client/
```

Auth Service additionally contains security/. Ride Service contains clients for Driver and Payment. Controllers remain thin; business rules belong in services; repositories handle persistence; DTOs form API boundaries; client packages contain inter-service calls.

18. Local Development Ports

Component	Port
API Gateway	8080
Ride Service	8081
Driver Service	8082
Payment Service	8083
Auth Service	8084
Eureka Server	8761
Node realtime-service	3000
React/Vite	5173
MySQL	3306

19. Spring Boot Dependency Intent

Component	Main Dependencies
Eureka	Spring Boot, Eureka Server, Actuator
Gateway	Spring Cloud Gateway, Eureka Discovery Client, Spring Security, JWT, Actuator
Auth	Spring Web, Security, JPA, MySQL Driver, Validation, JWT, Eureka Client, Actuator, Lombok, Mail
Ride	Spring Web, JPA, MySQL Driver, Validation, Eureka Client, OpenFeign, Actuator, Lombok
Driver	Spring Web, JPA, MySQL Driver, Validation, Eureka Client, OpenFeign, Actuator, Lombok
Payment	Spring Web, JPA, MySQL Driver, Validation, Eureka Client, Actuator, Lombok

Before implementation, verify the exact Spring Cloud release train compatible with Spring Boot 3.5.16. Do not invent or mix incompatible Spring Cloud versions.

20. Review 1 Scope

- Main frontend pages complete.
- Registration/login and JWT authentication complete.
- API Gateway complete.
- Eureka Server complete.
- All identified microservices created and registered.
- Service discovery demonstrated.
- Microservice skeletons follow the agreed package structure.
- At least one meaningful inter-service communication flow demonstrated.
- MySQL connectivity demonstrated for implemented services.
- Basic end-to-end request path works through the Gateway.

The strongest Review 1 demonstration should show the architecture working rather than only showing empty service folders.

21. Review 2 Scope

- Complete passenger booking workflow.
- Driver availability and matching.
- Driver accept/reject workflow.
- Driver arrival/start/complete workflow.
- Fare calculation.
- Payment creation and status handling.
- Real-time ride and driver updates.
- Role-based access control.
- Ride/payment history.
- Admin management screens.
- Inter-service failure/error handling.
- Automated and integration tests.
- Load balancing/service discovery demonstration.
- Dockerized deployment and production-oriented configuration.

22. Testing Strategy

Level	Focus
Unit tests	Fare calculation, state transitions, matching logic, service methods
Controller/API tests	Validation, HTTP status codes, authorization, response DTOs
Repository/integration	JPA mappings and MySQL persistence
Service integration	Ride -> Driver and Ride -> Payment communication
Gateway tests	Routing and security behavior
Real-time tests	WebSocket connection and event delivery
Frontend tests	Routes, protected pages, important user interactions
End-to-end	Passenger booking through payment completion

Every non-trivial business rule should have tests before considering that part of the implementation complete.

23. Error Handling

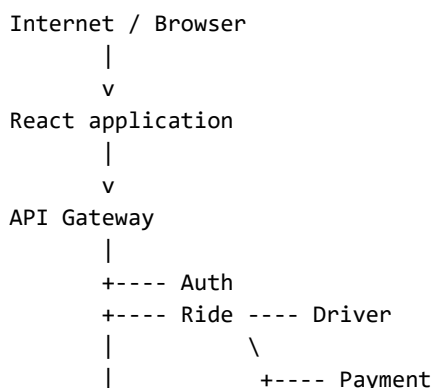
- Use consistent API error responses.
- Validate request DTOs at API boundaries.
- Use domain-specific exceptions for business failures.
- Handle not-found, invalid-state, unauthorized, forbidden, conflict, and downstream-service failures explicitly.
- Do not expose stack traces or sensitive internal information to clients.
- Inter-service calls should have timeouts and controlled failure behavior.
- A driver service failure must not leave the ride workflow in an unexplained state.

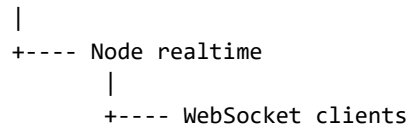
24. Load Balancing and Resilience

Eureka enables service discovery and allows multiple instances of business services to register under the same service name. Gateway/service clients can then route requests through discovered instances.

- Start with one instance per service for development.
- Demonstrate multiple instances after the core workflow is stable.
- Verify service registration and deregistration.
- Test that requests can reach multiple service instances.
- Add sensible timeouts and error handling for internal calls.

25. Deployment Architecture





MySQL databases

Eureka registry

Docker Compose is the initial deployment target because it provides a reproducible multi-service environment. Production deployment can later separate services, databases, networking, secrets, monitoring, and scaling.

26. Configuration and Secrets

- Never hard-code JWT secrets, database passwords, email credentials, or payment secrets in source code.
- Use environment variables/configuration files appropriate to the deployment environment.
- Keep local development configuration separate from production configuration.
- Use service names rather than hard-coded internal IP addresses when service discovery is available.

27. AI-Assisted Development Strategy

Development will use controlled AI/vibe coding rather than one giant autonomous prompt.

22. Freeze architecture and design before coding.
23. Create the repository structure.
24. Implement one bounded component at a time.
25. Ask the coding agent to inspect the current project before changing it.
26. Give the agent one clearly defined task with explicit files, constraints, and acceptance criteria.
27. Run the application/tests after each meaningful change.
28. Inspect the diff before accepting changes.
29. Commit small, coherent changes.
30. Move to the next task only after the current task works.

The AI agent must not invent new services, replace the agreed architecture, add major dependencies, rewrite unrelated files, or change frozen contracts without explicit approval.

28. Recommended Implementation Sequence

31. Create GoRyde root repository and directory structure.
32. Create and validate Eureka Server.
33. Create API Gateway and verify Gateway -> Eureka routing.
34. Create Auth Service, MySQL schema, registration, login, JWT, and /me.
35. Create Driver Service and MySQL connectivity.
36. Create Ride Service and MySQL connectivity.
37. Create first Ride -> Driver inter-service call.
38. Create Payment Service and MySQL connectivity.
39. Create Ride -> Payment inter-service call.
40. Complete Review 1 integration path.
41. Implement complete driver matching and ride state transitions.
42. Implement passenger and driver frontend workflows.

- 43. Implement payment workflow.
- 44. Implement Node.js real-time service and WebSocket events.
- 45. Connect live tracking/status updates to React.
- 46. Add admin functionality.
- 47. Add unit, integration, API, frontend, WebSocket, and end-to-end tests.
- 48. Demonstrate service load balancing with multiple instances.
- 49. Dockerize and validate the complete system.
- 50. Prepare Review 2 demonstration and final documentation.

29. Definition of Done

- The feature works through the intended public API.
- Authorization and ownership checks are correct.
- Persistence works against the service's own database.
- Required inter-service communication works through service discovery.
- Errors are handled consistently.
- Relevant tests pass.
- Frontend behavior matches the API contract.
- Real-time behavior is verified where applicable.
- No unrelated files were changed.
- The implementation follows the frozen architecture.

30. Final Architecture Decision Summary

Decision	Final Choice
Architecture	Microservices
Backend	Spring Boot 3.5.16 / Java 17
Frontend	React 19 + Vite + Tailwind CSS
Gateway	Spring Cloud Gateway
Discovery	Eureka
Database	MySQL with service-owned schemas
Internal communication	OpenFeign + Eureka
Authentication	JWT + Spring Security
Real-time	Node.js + WebSocket
Ride orchestration	Ride Service
Driver matching	Nearest available suitable driver
Fare	Base + distance + duration
Initial deployment	Docker Compose
Development method	Incremental AI-assisted implementation with review/test checkpoints

31. Frozen Scope for Implementation

The above architecture and contracts form the baseline for implementation. Any later architectural change should be treated as an explicit design decision rather than an incidental change made by an AI coding agent.